hashlock.

Security Audit

Klima Protocol (DeFi)

March 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

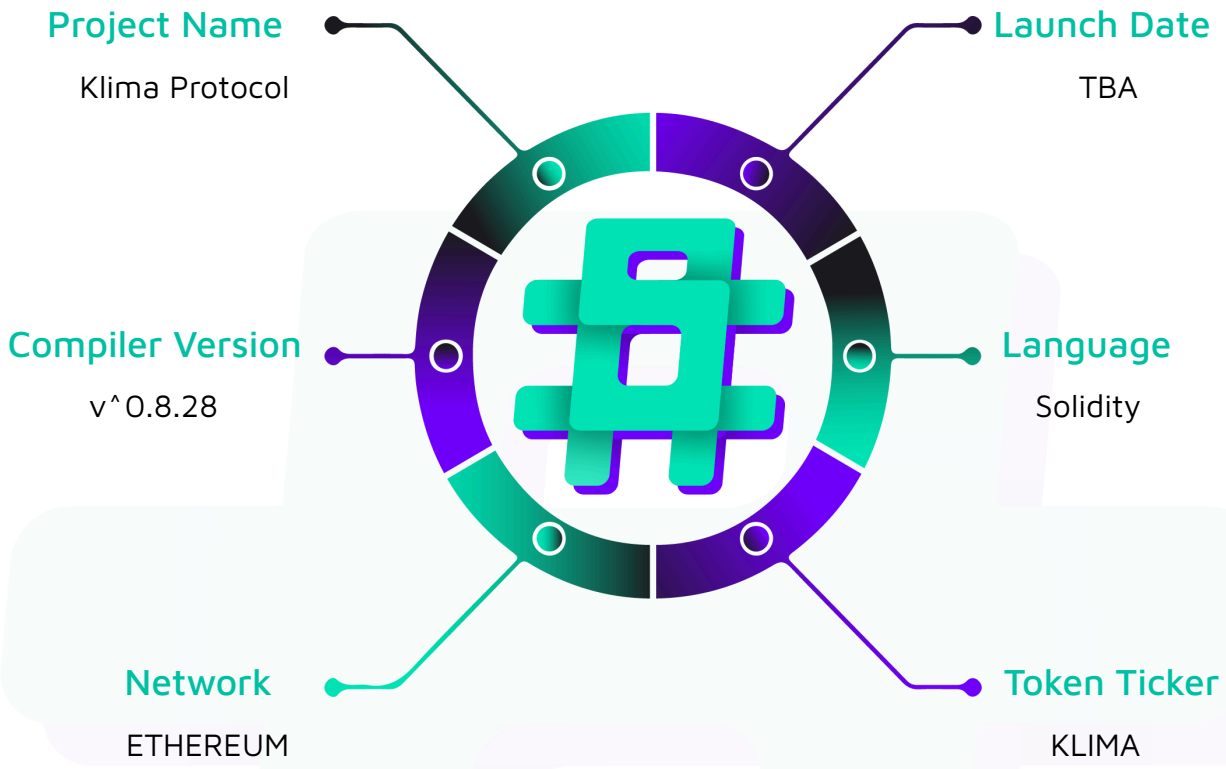
The Klima Protocol team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Klima Protocol represents a decentralized liquidity hub for carbon credits. It enables efficient carbon credit trades, generates fees, and facilitates and incentivizes market stakeholder participation through $KLIMA and $KLIMAX tokens.
Klima Protocol features a decentralized fee distribution where 100% of profits are allocated to ecosystem users, a dual token system in which $KLIMA represents carbon-backed assets and $KLIMAX governs emissions and liquidity incentives, an Autonomous Asset Manager (AAM) that optimizes carbon trading, pricing, and portfolio management, an incentive market that leverages staking maturities to create a yield curve and stabilize pricing, and carbon standardization that establishes uniform carbon assets and predictable delivery cycles. By integrating DeFi with carbon finance, KlimaDAO 2.0 enhances liquidity, transparency, and efficiency in global carbon markets.

Project Name: Klima Protocol
Project Type: DeFi, Token
Compiler Version: ^0.8.28
Website: https://www.klimaprotocol.com/
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the Klima Protocol, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Klima Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	March, 2025
Contract 1	KlimaFairLaunchStaking.sol
Contract 1 MD5 Hash	769ab3cc7da67e2ab3528044637d0597
Contract 2	KlimaFairLaunchBurnVault.sol
Contract 2 MD5 Hash	9913a422e473f0094a64fc7bba741e00
Contract 3	KlimaFairLanchPolygonBurnHelper.sol
Contract 3 MD5 Hash	bd17f7a08deda0ec6cfd83e789f40bb173fa9592
GitHub Commit Hash	3e9ce693115080ee1c3ca533ee38fdfc0d066fa7

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 Gas Optimisations

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
KlimaFairLaunchStaking.sol - Main staking contract used to transfer KLIMA_V0 give back KLIMAX and KLIMA - Time based staking and unstaking - Keeps track of amount staked and the length it was staked for - Allows users to unstake before freezeTime, cutting off the organic points	**Contract achieves this functionality.**
KlimaFairLaunchBurnVault.sol - Main vault that received KLIMA_V0 tokens that will be later burned cross chain by using Axelar Network - Has emergency withdrawal option	**Contract achieves this functionality.**
KlimaFairLanchPolgonBurnHelper.sol - Actual contract that executes Axelar's call and performs token burn	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Klima Protocol, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Klima Protocol smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] KlimaFairLaunchStaking#storeTotalPoints - An attacker can cause permanent DOS by staking multiple times, bricking the finalization

Description

The contract is vulnerable to a denial-of-service attack during the finalization process. An attacker can create numerous small stakes to deliberately exceed gas limits when their account is updated, permanently halting the finalization process.

Vulnerability Details

The vulnerability exists due to three key design decisions :

1. Unlimited Staking: The contract allows unlimited stake operations per user with no minimum stake size requirement

```
function stake(uint256 amount) public whenNotPaused {
    require(amount > 0, "Amount must be greater than 0");
    // No limit on number of stakes per user
    userStakes[msg.sender].push(newStake);
}
```

2. Monolithic User Processing: The `_updateUser()` function must process ALL of a user's stakes in a single transaction

```
function _updateUser(address _user) internal {
    _updateOrganicPoints(_user);  // Loops through ALL stakes
    _updateBurnDistribution(_user); // Loops through ALL stakes again
}
```

3. Sequential Finalization: The finalization process must process users in order, without the ability to skip problematic accounts

```
function storeTotalPoints(uint256 batchSize) public onlyOwner beforeFinalization {
    // Cannot skip users or partially process a single user
    for (uint256 i = finalizeIndex; i < endIndex; i++) {
        address staker = stakerAddresses[i];
        _updateUser(staker); // If this fails, finalizeIndex doesn't update
    }
    finalizeIndex = endIndex; // Only updates if all users processed successfully
}
```

This combination allows an attacker to create hundreds or thousands of small stakes, which would require gas exceeding block limits to process during finalization.

Proof of Concept

Paste the following POC in `KlimaFairLaunchStaking.t.sol`. Also add 3 more users and provide them with an initial balance.

```
function testDOSFinalization() public {

        vm.startPrank(owner);

        uint256 startTime = block.timestamp + 1 days;

        staking.enableStaking(startTime);

        vm.stopPrank();

        vm.warp(startTime);

        createStake(user1, 100e12);

        createStake(user2, 100e12);

        createStake(user3, 100e12);

        vm.startPrank(user3);

        uint256 stakeAmount = 10e12;

        IERC20(KLIMA_V0_ADDR).approve(address(staking), 1000e12);

        stakeMultipleTimes(user3, 1000);

        vm.stopPrank();

        createStake(user4, 100e12);
```

```
        createStake(user5, 100e12);

        vm.warp(block.timestamp + 100 days);

        uint256 gasBefore = gasleft();

        vm.startPrank(owner);

        staking.storeTotalPoints(5);

        uint256 gasAfter = gasleft();

        uint256 gasConsumed = gasBefore - gasAfter;

        console.log("gas consumed", gasConsumed); // 34635797

    }
function stakeMultipleTimes(address user, uint256 count) public {
        // Use a minimal stake amount to maximize number of stakes created

        uint256 tinyStakeAmount = 1e6; // Very small amount


        for (uint256 i = 0; i < count; i++) {

            // Each stake creates additional entries in the user's stake array

            staking.stake(tinyStakeAmount);


            // Optional: Add minor time variation to simulate real-world scenario

            // Also helps ensure each stake has slightly different timestamps/points

            vm.warp(block.timestamp + 1 minutes);

        }

    }
```

Impact

If triggered, the finalization process becomes permanently stuck, preventing the protocol's token migration from completing.

Users whose accounts were processed before the attack would have their `KLIMA_V0`

tokens burned but would be unable to claim new tokens.
Users after the attack point would be unable to participate in the migration at all.

Recommendation

Limit the number of stakes done by a single user.

Status

Resolved

Medium

[M-01] KlimaFairLaunchStaking#calculateBurn - Rounding error causes less burn calculation than intended

Description

The `calculateBurn` function in `KlimaFairLaunchStaking` has a rounding error in the time-based burn calculation that results in significantly less tokens being burned than the economic model intends, particularly for unstaking events that occur within the first ~116 hours of staking or at partial hour intervals.

Vulnerability Details

The vulnerability exists in the time-based burn calculation:

```
function calculateBurn(uint256 amount, uint256 stakeStartTime) public view returns (uint256) {
    // Base burn is 25% of amount
    uint256 baseBurn = (amount * 25) / 100;

    // If we're still in pre-staking period, only apply base burn
```

```
    if (block.timestamp < startTimestamp || stakeStartTime > block.timestamp) {
        return baseBurn;
    }
    // Calculate time-based burn percentage with higher precision (capped at 75%)
    uint256 timeBasedBurnPercent = (((block.timestamp - stakeStartTime) / 1 hours) * 75)
/ (24 * 365);
    if (timeBasedBurnPercent > 75) timeBasedBurnPercent = 75;
    // Calculate time-based burn amount
    uint256 timeBasedBurn = (amount * timeBasedBurnPercent) / 100;
    // Return total burn
    return baseBurn + timeBasedBurn;
}
```

`(block.timestamp - stakeStartTime) / 1 hours` truncates any partial hours. This means that if a user unstakes after 1 hour 59 minutes, this counts only as 1 hour.

Also, due to integer division by `(24 * 365) = 8760` will truncate the calculation to 0 for as long as 116 hours after `startTimestamp` which roughly equates to 5 days.

Proof Of Concept

Paste the following test in `KlimaFairLaunchStaking.sol`

```
   function test_RoundingInBurnCalculation() public {
        // stake some amount , wait until start timestamp , just before 116 hour unstake
, unstake and only burn 25 percent
        vm.startPrank(owner);
        uint256 startTime = block.timestamp + 1 days;
        staking.enableStaking(startTime);
        vm.stopPrank();
```

```
        vm.warp(startTime);

        vm.startPrank(user1);

        uint256 stakeAmount = 100 * 1e12;

        IERC20(KLIMA_V0_ADDR).approve(address(staking), 1000e12);

        staking.stake(stakeAmount);

        vm.warp(startTime + 116 hours);

        staking.unstake(stakeAmount);

}
```

You can see in the log how even after 116 hours, the final burn amount is only 25% of the original staked amount that is 100e12.

```
 emit StakeBurned(user: user1: [0x29E3b139f4393aDda86303fcdAa35F60Bb7092bF],
burnAmount: 25000000000000 [2.5e13], timestamp: 1742316757 [1.742e9])
```

Impact

Substantial reduction in tokens burned during early unstaking periods and at partial-hour intervals.

Sophisticated users can time their unstaking to occur just before hour boundaries to minimize burn penalties. For large holders, the impact is especially severe.

Recommendation

Implement a more precise time-based burn calculation. Make the calculation in seconds for improved precision.

Status

Resolved

Low

[L-01] KlimaFairLaunchBurnVault - Beware of block reorg

Description

There exists a scenario, where initiated burn is reversed in case of block reorganization on Polygon.

Recommendation

Wait for at least 15 blocks for confirmation of the burn.

Status

Acknowledged

[L-02] KlimaFairLaunchStaking - View function won't work if length of staker array is very long.

Description

`getTotalPoints()` iterates over every staker address and also their individual stakes. If the length of this array becomes large. It might result in DOS and the function won't work.

Recommendation

Implement a batch approach to query total points.

Status

Acknowledged

Gas

[G-01] KlimaFairLaunchStaking# - Only update points is amount < 0

Description

There is no need to update those stakes whose amount is zero.

Recommendation

Skip this process if amount is < 0.

Status

Resolved

[G-02] KlimaFairLaunchStaking - No need to update if burnRatio is not different

Description

In `_updateBurnDistribution` there is no need to update the user's stake if `burnRatio` is not different. This will save gas.

Recommendation

Skip if `burnRatio` is not different

Status

Resolved

QA

[Q-01] KlimaFairLaunchStaking Enhanced Documentation

Description

While functional, the contract's current documentation lacks sufficient detail for developers, auditors, and operators to fully understand implementation details and economic implications.

Recommendation

Consider adding code comments over various places like this :

```
// @notice Calculates the amount of tokens to burn when unstaking
// @param amount Amount of tokens being unstaked
// @param stakeStartTime Timestamp when the stake was created
// @return burnAmount Amount of tokens to burn
// @dev Burn formula: base 25% + time-based up to 75% (capped at 100% total)

function calculateBurn(uint256 amount, uint256 stakeStartTime) public view returns
(uint256 burnAmount) {
       // Code...
}
```

Status

Resolved

Centralisation

The Klima Protocol values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Klima Protocol seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au